

2026 年 7 月 9 日

已通读豆包 AI 翻译的文档，仍有一些不解之处，例如：

1. 第 6 部分的常驻空间上界 $N + (2r - 1) + 2(r - 1)B + B = N + O(rN^{1/r})$ 的这个系数 2 是什么
2. 下标访问原生 $O(r)$ 最坏时间是为什么，按我的理解顶多是二级指针， $O(1)$ 复杂度
3. 第 6.2 部分信用额度是什么鬼第一次听说，再看看
4. 第 8 部分看得有点云里雾里的，怀疑 AI 有删减

此外，还有一种局部看得懂，整合起来又有点晕晕的奇妙感觉

大致搞懂了它的数据结构构造（大成功？），复杂度分析也大致明白了

趣事：第 6.1 部分推论 6.2 中我还在想 $r = \log N$ 时常驻空间为什么不是 $O(\log N \cdot N^{1/\log N})$ ，突然意识到 $N^{1/\log N}$ 是个常数

以下为 AI 翻译的文档：

最优可变长数组

Robert E. Tarjan[†] Uri Zwick[‡]

2026 年 7 月 9 日

摘要

可变长数组支持在数组尾部（或两端）增删元素，同时能够根据下标常数时间访问任意元素。由于数组元素数量会动态变化，要实现空间高效的可变长数组，必须配套动态内存管理机制。经典**倍增扩容**方案可实现规模为 N 的数组仅占用 $O(N)$ 空间，且单次操作均摊时间 $O(1)$ ，甚至最坏时间 $O(1)$ 。Sitariski、Brodnik 等人提出了更优方案：存储规模 N 的数组仅需 $N + O(\sqrt{N})$ 空间，单次操作仍保持 $O(1)$ 时间；Brodnik 等人还给出证明，该空间复杂度是理论下界。

本文区分两类空间开销：一是数组常态存储、元素访问所需的常驻空间；二是扩容/缩容操作过程中临时占用的额外空间。对任意整数 $r \geq 2$ ，我们证明：若扩容、缩容时可短暂使用 $N + O(N^{1-1/r})$ 临时空间，则常态存储规模 N 的数组仅需 $N + O(N^{1/r})$ 常驻空间；下标访问最坏时间 $O(1)$ ，增删操作均摊时间 $O(r)$ 。

我们通过一套抽象**扩容博弈**模型做精确分析，证明：若某类通用数据结构仅允许 $N + O(N^{1/r})$ 常驻存储空间，即便只支持尾部添加与下标查询，尾部新增操作的均摊代价下界也为 $\Omega(r)$ 。除非 $r = 2$ ，否则增删操作无法做到最坏时间 $O(1)$ 。

1 引言

数组与可变长数组是应用最广泛的数据结构。令人意外的是，本文将证明，关于它们仍存在大量未被充分挖掘的理论结论。我们提出一套简洁的全新可变长数组实现方案，多数场景下内存利用率优于此前所有设计。本文视角偏向理论，但给出的实现方案也具备实际工程价值。

规模为 N 的数组是存储序列 $a_0, a_1, \dots, a_{N-1}$ 的抽象数据结构：对任意 $0 \leq i < N$ ，下标 i 对应元素可在常数时间读取/修改。数组最基础的实现方式是连续内存块，每块内存存储一个元素或元素指针，依托机器随机访问能力实现常数时间读写；但原生数组的容量固定。

可变长数组（又称动态数组、动态表）允许数组容量动态增减，同时保证任意下标元素可常数时间读写。单纯缩小数组很简单：只需不再使用已分配内存的后半段，但这种方式内存利用率极低；扩容则更棘手——数组连续内存块末尾的内存大概率已被其他数据占用，因此可变长数组必须依赖动态内存管理。

为简化讨论，本文大部分内容仅讨论**单端可变长数组**（仅在数组尾部增删元素）：数组规模从 N 增至 $N+1$ 时，在末尾新增元素 a_N ；规模从 N 缩减至 $N-1$ 时，丢弃末尾元素 a_{N-1} 。这类可变长数组是栈的泛化结构。本文所有结论均可拓展至两端均可增删的双端可变长数组（即双端队列 deque）：只需用两个单端可变长数组拼接实现即可。

教材中经典的可变长数组实现是**倍增扩缩容**（见 Cormen 等人《算法导论》17.4 节）：初始分配一块定长内存；内存填满时，分配一块两倍大小的新内存，将全部元素拷贝过去并释放旧内

存；当内存使用率低于 $1/4$ 时，分配一半大小的新内存拷贝元素、释放旧内存。均摊分析可证明，单次扩容/缩容操作均摊代价 $O(1)$ ，元素读写最坏时间 $O(1)$ ；借助后台渐进重构技术，甚至能将扩缩容优化为最坏时间 $O(1)$ 。

维基百科记载，Java `ArrayList`、Python `PyListObject`、C++ `STL vector` 等主流容器均采用几何扩缩容策略，扩容系数取 1.25^2 。

倍增方案仅保证 $O(N)$ 空间复杂度，虽能满足多数场景，但纯扩容场景下最多会浪费 50% 已分配内存。更一般地，若扩容系数为 $1 + \alpha$ ，内存闲置比例可达 $\frac{\alpha}{\alpha+1}$ ；若缩小 α 降低闲置占比，单次扩容均摊代价会上升至 $O(\frac{1+\alpha}{\alpha})$ 。

一个核心问题：能否构造仅占用 $N + o(N)$ 空间、且所有操作 $O(1)$ 时间的可变长数组？Sitariski 与 Brodnik 等人给出肯定答案：设计实现仅需 $N + O(\sqrt{N})$ 空间，我们将在第 3 节详细介绍。Brodnik 等人同时证明该方案存在下界——任意可变长数组实现，在某些时刻必然占用 $N + \Omega(\sqrt{N})$ 空间，第 4 节会复述其简洁证明。

本文核心创新：区分**常驻存储空间**（常态存放数组、支持下标读写所需空间）与**扩缩容临时空间**（扩容/缩容过程中短暂占用的额外内存）。很多场景下，我们愿意牺牲少量临时内存，换取常态存储更高的紧凑度；例如程序中存在大量可变长数组，但同一时刻仅一个数组执行扩缩容。

我们首个突破 Brodnik 等人下界的实现方案：常态存储仅 $N + O(N^{1/3})$ 空间，仅在扩缩容时临时占用 $N + O(N^{2/3})$ 空间；元素读写最坏时间 $O(1)$ ，增删均摊 $O(1)$ 。同时我们证明该方案渐近最优：任何仅用 $N + O(N^{1/3})$ 常驻空间的实现，扩缩容时必然需要 $N + \Omega(N^{2/3})$ 临时空间。

更一般地，对任意 $r \geq 2$ ：

1. 常态存储开销： $N + O(N^{1/r})$ ；
2. 扩缩容临时开销： $N + O(N^{1-1/r})$ ；
3. 下标访问最坏时间 $O(1)$ ；
4. 尾部增删操作均摊时间 $O(r)$ 。

由于无法使用后台重构技术，除非 $r = 2$ ，否则 $O(r)$ 的均摊代价无法优化为最坏时间 $O(1)$ 。 $r = 2$ 对应 Sitariski、Brodnik 的经典方案。

1.1 论文章节结构

1. 第 2 节：精确形式化问题定义与计算模型；
2. 第 3 节：梳理前人相关工作，重点介绍 Sitariski 的 HAT 结构、Brodnik 等人的方案；
3. 第 4 节：复述 Brodnik 的下界证明，并给出我们的拓展下界；
4. 第 5 节：给出 $r = 3$ 的完整构造（常驻 $O(N^{1/3})$ 、临时 $O(N^{2/3})$ ）；
5. 第 6 节：对任意 $r \geq 2$ 给出通用无递归实现，证明增删均摊 $O(r)$ ；
6. 第 7 节：提出一套结构变换技巧，将常驻空间从 $O(rN^{1/r})$ 优化至 $O(N^{1/r})$ ，同时不破坏常数访问、 $O(r)$ 均摊增删；

7. 第 8 节：抽象出**扩容博弈**模型并完整求解，用于证明下界；
8. 第 9 节：依托扩容博弈，证明仅使用 $N + O(N^{1/r})$ 常驻空间时，尾部新增操作均摊代价下界 $\Omega(r)$ ；
9. 第 10 节：总结与展望。

2 可变长数组形式化定义

可变长数组作为抽象数据类型，支持以下操作：

1. `Array()`：创建空数组并返回；
2. `A.Length()`：返回数组当前元素个数；
3. `A.Get(i)`：返回下标 i 对应的元素 a_i ，要求 $0 \leq i < A.Length()$ ；
4. `A.Set(i, a)`：将下标 i 元素修改为 a ，要求 $0 \leq i < A.Length()$ ；
5. `A.Grow(a)`：数组长度 $+1$ ，末尾新增元素 a ；
6. `A.Shrink()`：数组长度 -1 ，丢弃末尾元素。

本文仅允许在数组尾部增删元素。若支持任意位置插入删除（并同步平移后续元素下标），问题难度会大幅提升；此时所有操作最优时间界为 $O(\frac{\log N}{\log \log N})$ ，且该界紧（Dietz、Fredman&Saks）。另有一类方案：下标访问最坏 $O(r)$ ，插入删除均摊 $O(N^{1/r})$ （Goodrich、Joannou 等人）。再次强调：本文仅讨论尾部增删场景。

我们假设内存管理器支持分配、释放任意长度的定长数组（类比 C/C++ 的 `malloc/free`），简化起见，单次分配/释放操作代价为常数；即便分配长度 N 的内存块代价为 $O(N)$ ，本文所有均摊复杂度结论依然成立。本文不讨论内存管理器底层实现。

可变长数组实现由若干动态分配的定长内存块（简称块）组成，分为三类：

1. 数据块：存储数组元素，每个元素占一个机器字；
2. 索引块：存储指向其他块的指针，每个指针占一个机器字；
3. 辅助块：存放数据结构元信息，如各块长度。

本文所有实现均遵循该框架。若数组当前规模为 N ，则全部数据块总容量至少为 N （第 4 节定理 4.1 证明后补充讨论）。除主索引块外，所有块必须被某索引块内的指针引用，否则无法访问。数据结构总空间为所有已分配块的长度之和。

定义 2.1 ($(s(N), t(N))$ 实现). 设 $s(N), t(N)$ 为单调不减函数。若某可变长数组满足：

1. 存储规模 N 的数组时，常驻总空间不超过 $N + s(N)$ ；
2. 执行扩容/缩容操作过程中，瞬时总空间不超过 $N + t(N)$ ；

则称其为 $(s(N), t(N))$ 实现。

3 前人相关工作

设计空间高效可变长数组属于紧凑数据结构领域，Raman、Munro 等人已有大量综述，但仅有少数文献专门研究本文定义的单端可变长数组。

3.1 基础单块实现

基础实现仅用一块连续定长内存承载数组：内存填满时，分配更大内存、拷贝全部元素、释放旧内存；内存空置过多时，分配更小内存拷贝元素。实现可自由选择新内存块的大小。

1. 极致省常驻空间方案：每次增删都重新分配恰好等于元素数量的内存。常驻空间 $N + O(1)$ （仅存长度与内存指针），但单次增删均摊代价 $\Omega(N)$ ；扩缩容时需同时保存新旧两块内存，临时空间 $2N + O(1)$ ，仅适合极少扩缩容的场景。
2. 几何扩缩容（工程主流）：固定参数 $\alpha > 0$ ，内存填满时分配 $(1 + \alpha)N$ 大小新块；内存元素仅占当前块容量 $\frac{1}{(1 + \alpha)^2}$ 时，分配 $\frac{N}{1 + \alpha}$ 大小新块拷贝元素。简单计算可得：扩容均摊代价 $\frac{1 + \alpha}{\alpha}$ ，缩容均摊代价 $\frac{1}{\alpha}$ 。合理选取 α 可平衡空间与时间，适配绝大多数工程场景。

3.2 Sitarski 的哈希数组（HAT）

Sitarski 提出名为 HAT（哈希数组树）的结构，但命名存在误导——不含哈希，本质分层链表。

HAT 维护长度为 B 的索引块，满足 $\sqrt{N} \leq B < 4\sqrt{N}$ ；配套 $\lceil N/B \rceil$ 或 $\lceil N/B \rceil + 1$ 个尺寸 B 的数据块。 N 个元素顺序存入前 $\lceil N/B \rceil$ 块； N 无法整除 B 时最后一块半满；存在 $\lceil N/B \rceil + 1$ 块则为空。索引块第 i 项存储第 i 个数据块指针。

总常驻空间： $N + 3B + 2 = N + O(\sqrt{N})$ 。 $3B$ 为索引块 + 两块闲置数据块开销。下标 i 元素位于第 $\lfloor i/B \rfloor$ 块、偏移 $i \bmod B$ ，读写最坏 $O(1)$ ； B 取 2 的幂时可通过位运算快速计算下标。

扩容逻辑： $\sqrt{N} < B$ 即 $N < B^2$ 时，末尾块未满足直接写入；已满但存在空块则写入空块；无空块则分配新 B 块。分配代价常数则扩容最坏 $O(1)$ ；分配代价 $O(B)$ 时扩容均摊 $O(1)$ 。

满载 $N = B^2$ 时将 B 翻倍重构；初始 B 为 2 的幂则重构后不变。重构 $O(N)$ 拷贝代价分摊至 $\Omega(N)$ 次扩容，均摊 $O(1)$ 。

缩容对称：仅当数组元素总量降至 $B^2/16$ （ $B = 4\sqrt{N}$ ）、末尾两块全空时才折半重构，避免频繁扩缩震荡。两次重构间至少 $\Omega(N)$ 次操作，缩容均摊 $O(1)$ 。

重构仅需 $O(\sqrt{N})$ 临时空间，新旧块逐块分配释放，无需完整保存两份数组。

可通过维护两种尺寸数据块 + 后台渐进重构，转化为最坏时间界版本，但结构复杂度大幅提升；下一节方案无需全局重构、元素全程不移动。

3.3 Brodnik 等人的分层块结构

Brodnik 独立提出分层方案，常驻 $N + O(\sqrt{N})$ ，扩缩容最坏 $O(1)$ （分配常数时间）；优势：无全局重构、元素不移动。两种变体：

1. 基础变体：数据块尺寸依次 $1, 2, 3, \dots$ ；前 k 块总容量 $\approx k^2/2$ ， $k = \lceil \frac{\sqrt{8N+1}-1}{2} \rceil$ ；末块半满，预留一块空块。索引块尺寸 $\Theta(\sqrt{N})$ ，自身朴素动态数组实现。有空位直接写入；无空位分配

新块，索引块满则扩容拷贝指针，后台拷贝实现最坏 $O(1)$ 。下标块编号 $\ell = \lceil \frac{\sqrt{8i+1}-1}{2} \rceil$ ，块内偏移 $i - \frac{\ell(\ell-1)}{2}$ ，读写 $O(1)$ 但需要开平方运算。

2. 无平方根变体：所有数据块尺寸为 2 的幂，引入多层超块结构，利用移位运算替代开平方定位下标，规避浮点与开方开销，整体空间复杂度依旧 $N + O(\sqrt{N})$ ，索引总开销仍为 $\Theta(\sqrt{N})$ 级别。

4 Brodnik 下界定理及其拓展

本节复述原始下界与分离空间拓展下界。

定理 4.1. 任意仅支持扩容、下标访问的可变长数组，在某些时刻必然占用 $N + \Omega(\sqrt{N})$ 空间， N 为当前元素总数。

证明. 完成 N 次扩容后设使用 k 块连续内存；全部数据块容量至少 N ，每块需一条指针，指针总开销 k 。

- 若 $k \geq \sqrt{N}$ ：总空间 $\geq N + \sqrt{N}$ ，下界成立；
- 若 $k < \sqrt{N}$ ：必存在尺寸 $\ell > \sqrt{N}$ 的块。该块在数组规模 $N' \leq N$ 时分配，分配瞬间总空间 $N' + \ell > N' + \sqrt{N'}$ ，下界成立。

证毕。 □

下界两条基础假设：

1. N 个独立 w 比特元素至少占用 N 机器字；
2. k 条指针占用 k 机器字（地址空间 2^w ）。

地址空间 2^{cw} ($0 < c < 1$) 时指针开销为 ck ，仅常数变化，不改变渐近阶。

定理 4.2 ($(s(N), t(N))$ 分离下界). 任意仅支持扩容、访问的 $(s(N), t(N))$ 实现，满足 $s(N) \cdot t(N) \geq N$ 。

证明. N 个元素分散至多 $s(N)$ 块，鸽巢原理存在块尺寸 $\geq N/s(N)$ 。该块分配时刻瞬时额外空间 $\geq N/s(N)$ ； $t(N)$ 单调不减， $t(N) \geq N/s(N)$ ，变形得 $s(N)t(N) \geq N$ 。证毕。 □

推论 4.1. 对任意整数 $r \geq 1$ ，常驻仅 $N + O(N^{1/r})$ 的实现，扩容瞬时必有 $N + \Omega(N^{1-1/r})$ 额外空间。

后文证明该下界是紧的（存在实现刚好匹配该空间权衡）。

5 $r = 3$ 的极简实现: $(O(N^{1/3}), O(N^{2/3}))$ 方案

构造常驻 $O(N^{1/3})$ 、临时 $O(N^{2/3})$ ，访问最坏 $O(1)$ ，增删均摊 $O(1)$ 。

核心改造 HAT 结构：外层大块尺寸 $N^{2/3}$ 、一级索引开销 $N^{1/3}$ ；半满尾部块嵌套一层 HAT，嵌套层额外开销仅 $O((N^{2/3})^{1/2}) = O(N^{1/3})$ 。合并拷贝瞬时需要 $\Omega(N^{2/3})$ 临时内存，匹配推论下界。也可基于 Brodник 分层思路构造等价方案。

无递归完整描述：维护参数 $N^{1/3} \leq B \leq 4N^{1/3}$ ，分两类内存块：

1. 大块：尺寸 B^2 ，总数量约 N/B^2 ，所有大块永久填满；
2. 小块：尺寸 B ，最多允许 $2B$ 个，至多一块部分填充、至多一块完全闲置空块。

配套两级索引分别管理大块、小块；当累计 B 个完整小块时可合并为一块大块；无可用小块时拆分一块大块生成小块。所有访问最坏 $O(1)$ ，增删均摊 $O(1)$ 。

6 对任意 $r \geq 2$ 的通用 $(O(rN^{1/r}), O(N^{1-1/r}))$ 实现

泛化 $r = 3$ 构造，无递归分层设计，常驻额外 $O(rN^{1/r})$ ，扩容临时峰值 $O(N^{1-1/r})$ 。

设定参数 $N^{1/r} \leq B < 4N^{1/r}$ ，所有数据块尺寸为 B 的幂 $B, B^2, \dots, B^{r-1}$ 。仅扩容场景每层最多 B 块；同时支持增删时放宽至每层最多 $2B$ 块（冗余基数计数器，避免频繁震荡）。

- $B^2, \dots, B^{r-1}$ 尺寸块永久填满；
- 仅最末尾 B 尺寸块允许部分填充。

扩容逻辑：末尾 B 块有空位直接写入；已满且总数不足 $2B$ 则分配新 B 块；若达到 $2B$ 个完整小块，则合并 B 个小块生成高一阶大块，逻辑等价基数 B 进位计数器。

当总元素 $N = B^r$ 时，将参数 B 翻倍全局重构；配套 $r-1$ 层独立索引块。常驻空间上界 $N + (2r-1) + 2(r-1)B + B = N + O(rN^{1/r})$ ；合并 B^{r-1} 尺寸大块时临时空间峰值 $O(N^{1-1/r})$ 。

原生未优化时下标访问最坏 $O(r)$ ；6.3 节位运算优化后可达 $O(1)$ 最坏时间。增删均摊代价 $O(r)$ ：每个元素先存入 B 小块，逐层合并提升块阶，全局重构分摊后仅增加常数开销。

缩容适配：每层上限 $2B$ 块；达到 $2B$ 完整小块时合并；无可用小块则拆分最小阶大块，均摊代价依旧 $O(r)$ 。

6.1 实现伪代码说明

n_i 代表尺寸 B^i 的数据块数量， $A[i]$ 为对应索引块； n_0 为最后一块 B 块内已存元素数量。

6.1.1 Grow(a) 扩容流程

1. 若 $N = B^r$ ，执行 Rebuild($2B$)， B 翻倍重构；
2. 若 $n_1 = 2B, n_0 = B$ （所有 B 块填满），调用 Combine-Blocks 合并低层块；
3. 无空余位置则分配全新 B 块，重置 $n_0 = 0$ ；
4. 将新元素写入末尾 B 块空位， n_0, N 自增 1。

Combine-Blocks: 找到最小未满足层 k , 自底向上每层合并 B 个小块为一块高阶大块, 调整索引指针、释放旧内存块。

6.1.2 Shrink() 缩容流程

1. 若 $N = (B/4)^r$, 执行 Rebuild($B/2$), B 折半重构;
2. 若无 B 尺寸块, 调用 Split-Blocks 拆分大块;
3. 删除末尾元素, n_0, N 自减 1; 若 $n_0 = 0$, 释放空 B 块。

Split-Blocks: 找到最小非空大块, 拆分为多层小块, 分配新内存、拷贝元素、释放原大块。

辅助函数:

- Allocate(ℓ): 分配长度 ℓ 内存块, 返回指针;
- Deallocate(X): 释放内存块 X ;
- Copy(X, x, Y, y, ℓ): 从 X 偏移 x 拷贝 ℓ 个元素至 Y 偏移 y ;
- Rebuild(B'): 以新参数 B' 全局重构整套分层结构。

定理 6.1. 该实现为 $(O(rN^{1/r}), O(N^{1-1/r}))$ 可变长数组; 扩容、缩容均摊 $O(r)$, 下标读写最坏 $O(1)$ 。

推论 6.1. 取 $r = \log N$, 存在实现常驻额外 $O(\log N)$, 增删均摊 $O(\log N)$, 访问最坏 $O(1)$ 。

6.2 6.2 增删操作均摊代价证明 (引理 6.3)

定义单次操作代价为元素拷贝赋值次数。

1. 扩容: 基础写入代价 1; 每个底层元素分配 $r - 1$ 单位信用额度, 每向上合并一层消耗 1 信用, 合并操作均摊代价归零, 单次扩容基础均摊 $O(r)$;
2. 缩容: 每次缩容为所有层级增加固定信用; 拆分大块的拷贝开销由长期累积信用覆盖, 拆分均摊代价归零, 单次缩摊 $O(r)$;
3. 全局重构: 两次重构间至少执行 $\Omega(N)$ 次增删, 重构总拷贝 N 可完全分摊, 不提升渐近阶。

综上扩容、缩容均摊代价 $O(r)$ 。

6.3 6.3 $O(1)$ 下标访问优化 (引理 6.4)

假设 CPU 内置最高置位 msb 常数查询指令, 对结构小幅改造:

1. 令 $B = 2^b$ (2 的幂次);
2. 总元素 N 可表示冗余 B 进制: $N = \sum_{j=0}^{r-1} n_j B^j$, $0 \leq n_j \leq 2B$;
3. 每层块计数压缩存入两个机器字 N^0, N^1 , 移位与按位运算快速提取;

4. 预计算每层容量前缀和 N_k ，代表尺寸不超过 B^{k-1} 的总元素数量。

访问下标 i 步骤：

1. 计算相对末尾偏移 $x = N - 1 - i$ ；
2. 调用 `msb` 定位 x 最高进制位，确定目标块阶 k ；
3. 移位、按位与替代取模/除法，快速计算块内偏移，全程常数操作。

若无原生 `msb` 指令，现有算法耗时 $O(\log \log w)$ ，不改变渐近 $O(1)$ ；工程场景 r 通常极小，直接逐层遍历也具备实用性。

7 数据结构变换优化常驻空间

当 $r(N)$ 随 N 缓慢增长时，可将常驻额外空间从 $O(rN^{1/r})$ 降至 $O(N^{1/r})$ ，同时保留 $O(r)$ 均摊、 $O(1)$ 访问性能。

定义 7.1 (标准 $(s(N), t(N))$ 实现). 满足两条约束：

1. 分配新块时按顺序拷贝若干旧块内容，拷贝完成立刻释放旧块；
2. 拷贝操作结束后，常驻空间回归 $N + s(N)$ 级别。

本文全部实现均属于标准实现。

引理 7.1. 任意标准 $(s(N), O(N))$ 实现，可变换得到 $(s(N) + O(N/t(N)), s(N) + O(t(N)))$ 新实现，访问、均摊复杂度不变。

思路：超大块拷贝拆分多段，每段不超过 $t(N)$ ；新增指针开销 $O(N/t(N))$ ，瞬时空间峰值变为 $s(N) + O(t(N))$ 。

定理 7.1. 对满足 $r(N) \leq \frac{1}{2} \frac{\log N}{\log \log N}$ 的 r ，存在 $(O(N^{1/r}), O(N^{1-1/r}))$ 实现：访问最坏 $O(1)$ ，增删均摊 $O(r)$ 。

证明思路：对 $r' = 2r$ 的基础构造应用变换，取 $t(N) = N^{1-1/r}$ ；约束下 $rN^{1/(2r)} \leq N^{1/r}$ ，常驻开销简化为 $O(N^{1/r})$ 。

8 扩容博弈：下界抽象模型

为证明常驻空间受限场景下扩容均摊下界，抽象单人扩容博弈模型，刻画所有标准实现的合并行为。

8.1 (N, k, ℓ) 扩容博弈定义

参数：总插入 N 、最大子数组数量 k 、单块最大空位 ℓ 。初始 k 个空数组 $A_1 \dots A_k$ ；元素存储顺序倒置； a_i 为 A_i 内元素个数。约束：

1. 空数组全部集中在最左侧;
2. 仅首个非空数组允许保留至多 ℓ 空位, 其余全部填满;
3. 总闲置开销等价 $k + \ell$, 对应数据结构常驻额外空间。

单次插入三类操作:

1. 首块存在空位直接写入, 代价 1;
2. 无空位但存在空数组, 分配长度 $\ell + 1$ 新块存入元素, 代价 1;
3. 所有子数组填满: 选定 i , 合并前 i 块为新 A_i , 代价 $1 + \sum_{j=1}^i a_j$ 。

记 $C_{N,k,\ell}$ 为 N 次插入最小总代价; 单次均摊 $A_{N,k,\ell} = C_{N,k,\ell}/N$ 。

8.2 $\ell = 0$ 归约至无空位场景

引理 8.1. 若 N 可被 $\ell + 1$ 整除, 则 $C_{N,k,\ell} = (\ell + 1)C_{N,k,0}$, 单次均摊代价相等。

直观理解: 每填满 $\ell + 1$ 个空位等价完成一轮块填充, 可把每 $\ell + 1$ 次插入打包为 $\ell = 0$ 博弈单元。

8.3 $\ell = 0$ 博弈精确求解

记 $C_{N,k} = C_{N,k,0}$; 状态集合 $P_{N,k}$ 满足总元素 $\sum a_i = N$ 、空数组前置。

定理 8.1. 1. 若 $\binom{n+k-1}{k} - 1 \leq N \leq \binom{n+k}{k} - 1$, 总最小代价 $C_{N,k} = (N + 1)n - \binom{n+k}{k+1}$;

2. 区间内每次新增元素增量代价恒为 n : $C_{N,k} - C_{N-1,k} = n$;

3. 最优状态充要条件: $\binom{n+i-2}{i} \leq a_i \leq \binom{n+i-1}{i}, \forall i \in [k]$ 。

推论 8.1. 1. 当 $N = \binom{n+k}{k} - 1$ 时, 总代价 $C_{N,k} = \frac{kn}{k+1}(N + 1)$;

2. 区间内均摊代价下界 $A_{N,k} \geq \frac{1}{2}(n - 1)$ 。

8.4 最优策略: 二项式计数器

唯一最优插入序列对应二项式计数器; 维护计数 $b_1 \dots b_k$, 每次寻找最小 i 满足 $b_i < b_{i+1}$ 并合并进位; 全部 $b_i = n$ 时策略唯一。

8.5 博弈规则拓展

允许任意区间无序合并不会降低最小总代价; 拓展博弈完整覆盖本文所有块合并、拆分逻辑。

9 扩容操作均摊代价下界

定理 9.1. 任意标准可变长实现，常驻仅 $O(rN^{1/r})$ 、 $r = O(\log N)$ 时，扩容均摊代价下界 $\Omega(r)$ 。

证明思路：取博弈参数 $k = \Theta(rN^{1/r})$ ，利用二项式系数放缩证明博弈最优均摊至少线性于 r ；所有标准实现行为等价博弈合法操作，故数据结构下界为 $\Omega(r)$ 。

10 总结

本文提出一族分层可变长数组实现，完整刻画 ** 常驻空间、扩容临时空间、增删均摊时间 ** 三者最优权衡关系，同时保证 $O(1)$ 下标访问，从理论层面几乎完整解决该基础数据结构的时空权衡问题。

现有下界仅适用于本文定义的标准实现；引入元素、指针不可压缩假设后，下界可拓展至全部实现，无本质证明障碍。

扩容博弈具备独立理论价值；未来可探索无需完整闭式解的归纳式下界证明。

参考文献

- [1] Philip Bille, Anders Roy Christiansen, and Inge Li Gørtz. Fast dynamic arrays. In Proc. of 25th ESA, volume 87 of LIPIcs, pages 16:1–13, 2017.
- [2] Andrej Brodnik, Svante Carlsson, Erik D. Demaine, J. Ian Munro, and Robert Sedgwick. Resizable arrays in optimal time and space. In Proc. of 6th WADS, pages 37–48, 1999.
- [3] Andrej Brodnik, Peter Bro Miltersen, and J. Ian Munro. Trans-dichotomous algorithms without multiplication. In Proc. of 4th WADS, volume 1272 of Lecture Notes in Computer Science, pages 426–439. Springer, 1997.
- [4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to algorithms. MIT press, 3rd edition, 2009.
- [5] Paul F. Dietz. Optimal algorithms for list indexing and subset rank. In Proc. of 1st WADS, volume 382 of Lecture Notes in Computer Science, pages 39–46. Springer, 1989.
- [6] Michael L. Fredman and Michael E. Saks. The cell probe complexity of dynamic data structures. In Proc. of 21st STOC, pages 345–354. ACM, 1989.
- [7] Michael L. Fredman and Dan E. Willard. Surpassing the information theoretic bound with fusion trees. J. Comput. Syst. Sci., 47(3):424–436, 1993.
- [8] Michael T. Goodrich and John G. Kloss II. Tiered vectors: Efficient dynamic arrays for rankbased sequences. In Proc. of 6th WADS, pages 205–216, 1999.
- [9] Stelios Joannou and Rajeev Raman. An empirical evaluation of extendible arrays. In Proc. of 10th SEA, volume 6630 of Lecture Notes in Computer Science, pages 447–458. Springer, 2011.

- [10] Haim Kaplan, Robert E. Tarjan, Or Zamir, and Uri Zwick. Simulating a stack using queues. In Proc. of 33rd SODA, pages 1901–1924, 2022.
- [11] Jyrki Katajainen. Worst-case-efficient dynamic arrays in practice. In Proc. of 15th SEA, volume 9685 of Lecture Notes in Computer Science, pages 167–183. Springer, 2016.
- [12] Donald E Knuth. The Art of Computer Programming, Volume 4, Fascicle 1: Bitwise Tricks & Techniques; Binary Decision Diagrams. Addison-Wesley, 2009.
- [13] J. Ian Munro and S. Srinivasa Rao. Succinct representation of data structures. In Handbook of Data Structures and Applications, pages 595–610. Chapman and Hall/CRC, 2018.
- [14] Rajeev Raman, Venkatesh Raman, and S. Srinivasa Rao. Succinct dynamic data structures. In Proc. of 7th WADS, volume 2125 of Lecture Notes in Computer Science, pages 426–437. Springer, 2001.
- [15] Edward Sitarski. Algorithm alley: HATs: Hashed array trees. Dr. Dobbs’ s Journal of Software Tools, 21(9):107–11, September 1996.
- [16] Wikipedia contributors. Dynamic array - Wikipedia, the free encyclopedia, 2022.
- [17] Wikipedia contributors. Hashed array tree - Wikipedia, the free encyclopedia.

11 关键术语对照表

英文原文	标准中文译法
resizable array	可变长数组
dynamic array / vector	动态数组
amortized time	均摊时间
worst-case time	最坏时间
block	内存块
data block	数据块
index block	索引块
doubling technique	倍增扩容法
geometric expansion	几何扩缩容
HAT (Hashed Array Tree)	哈希数组树
succinct data structure	紧凑数据结构
lower bound / upper bound	下界 / 上界
cell probe model	单元探测模型
binomial counter	二项式计数器
growth game	扩容博弈
temporary space	临时空间
resident space	常驻空间
rebuild	全局重构
combine / split blocks	块合并 / 块拆分

以上为 AI 翻译的文档。

2026 年 7 月 9 日

第二次通读豆包 AI 翻译的文档（你问我为什么不去读原文？因为要做 AI review，重新读了一次）

第二次通读发现了一些问题（不知是 AI 译错还是写错），例如：

1. 第 1 部分引言中若扩容系数为 $1 + \alpha$ ，内存闲置比例可达 $\frac{\alpha}{\alpha+1}$ 而不是 $\frac{1}{\alpha+1}$
2. 第 3.2 部分总常驻空间依旧惊现神秘常数系数 2，推测为索引块指针占用（但为什么是 2 呢？）
3. 第 3.3 部分无平方根变体没有细讲，不知道复杂度怎么做到 $O(N + \sqrt{N})$ （我寻思新块的大小不是比之前所有块加起来还大吗）
4. 第 5 部分为什么小块至多为 $2B$ 个呢？我寻思小块有 B 个的时候不就可以合并成 B^2 的大块了吗

发现一些有意思的相关研究：

1. 第 2 部分中提到的支持任意位置插入删除数组实现（不是链表！？）

2. 第 3.2 部分双尺寸块 + 后台重构转为最坏时间版本

一些小小的思考:

1. 第 3.2 部分为什么 $N = \frac{B^2}{16}$ 才重构缩容而不是 $N = \frac{B^2}{4}$? 推测为为了保证缩容后增容的空间, 若 $N = \frac{B^2}{4}$ 就缩容, 那缩容后 $N = \frac{B^2}{4}$, 符合增容条件再次增容, 产生振荡
2. 第 3.2 部分重构的缩容增容的系数都是 B 乘 2 B 除 2 , 为什么依旧是神秘系数 2 ?

对第一次阅读问题的解决:

1. $N + (2r-1) + 2(r-1)B + B$ 的 $(2r-1)$ 系数 2 好像真是指针大小, $2(r-1)B$ 系数 2 好像是最多支持每层有 $2B$ 块具体而言该公式的组成如下: N : 有效数据占用 $2(r-1)$: 指向每一层的索引块的指针占用 $2(r-1)B$: 每一层的索引块占用 $B-1$: 冗余未使用的数据块大小
2. 第 3.2 部分重构的缩容增容的系数都是 B 乘 2 B 除 2 , 为什么依旧是神秘系数 2 ?

趣事: 第 6.1 部分推论 6.2 中我还在想 $r = \log N$ 时常驻空间为什么不是 $O(\log N \cdot N^{1/\log N})$, 突然意识到 $N^{1/\log N}$ 是个常数

本文 Latex 格式有 AI 辅助修正, 美化, 但内容不变。